VGI-SKT: Competition Solution for the 2026 BEHAVIOR Challenge

Team VGI-SKT
Visual & Geometric Intelligence Lab, Seoul National University SK Telecom

Abstract

The 2026 BEHAVIOR Challenge evaluates long-horizon mobile manipulation across 100 household tasks in realistic simulated home environments. This report presents the solution of Team VGI-SKT. We build on G0.5, a vision-language-action (VLA) model that combines autoregressive reasoning and action-token generation with a flow-matching action expert. Our adaptation proceeds in three stages. First, we fine-tune G0.5 on all 20,000 demonstrations across the 100 tasks using action supervision alone. Second, we post-train the model with Belief-Graph Chain-of-Thought *(BG-CoT), providing supervision for the current subtask and symbolic representations of object states, relations, expected effects and goal progress. These targets are constructed from BDDL task goals, skill annotations and simulator states. During post-training, we use* skill-balanced sampling *to draw training samples from rare skills more often, while limiting repeated exposure to individual frames. Third, we train* skill-specific LoRA adapters *on the action expert for six motion-centric skill groups; at inference time, the model's predicted subtask determines which adapter to activate. We also use a* hybrid action decoding *variant, in which arm actions are decoded from autoregressive action tokens, while torso, base and gripper actions are generated by the flow-matching action expert. Across the 100-task public evaluation suite, BG-CoT with FM-only decoding achieved the highest mean Q-score among the six individual configurations (0.360), compared with 0.298 for the 700k action-only supervised checkpoint. For the final submission, we selected one configuration per task using these public results. Aggregating the selected rollouts on the same instances yielded a mean Q-score of 0.445 and a full-task success rate of 22.3% over 1,000 episodes.*

1. Introduction

Vision-language-action (VLA) models have become a general approach to robot control, typically coupling a pretrained vision-language backbone with an action module and training on large, heterogeneous robot data [2–4, 6, 11, 12]. Completing a household activity requires the robot to execute a long sequence of skills such as navigating to a cabinet, grasping an object, and placing it at its destination, while keeping track of which subtasks it has completed. An episode can span thousands of control steps, allowing small action errors to accumulate and drive the robot into states rarely encountered in the demonstrations.

The BEHAVIOR Challenge evaluates long-horizon household manipulation in the BEHAVIOR-1K benchmark using the OmniGibson simulator [8]. The 2026 edition includes 100 household tasks, retaining all 50 tasks from the 2025 edition and adding 50 new ones. It provides 200 teleoperated demonstrations per task collected with the R1Pro bimanual mobile manipulator. The top two teams in the 2025 edition both built on $\pi_{0.5}$ [12]. The winning team used stage tracking and correlated flow-matching noise [7], while the runner-up, Team Comet, explored multitask training and rejection-sampling fine-tuning [1].

We build on G0.5 [10], which generates reasoning in natural language and discrete action tokens in a shared autoregressive stream and also supports continuous action generation through a flow-matching action expert. This shared stream architecture lets us train the model to reason about task state and condition action generation on the predicted subtask without a separate planner. Our method comprises three training stages, with skill-balanced sampling during post-training and hybrid action decoding as an inference variant (Fig. 1).

- **Action-only fine-tuning.** We fine-tune the official G0.5 checkpoint on all 20,000 demonstrations of the 100 challenge tasks using action supervision alone (Sec. 4.1).
- **BG-CoT post-training.** We construct training targets from BDDL task goals, skill annotations, and recorded simulator states. These targets supervise the prediction of the current subtask and a symbolic *belief graph*, including remaining goals, current object states, observed relations, and the expected effect of the next skill (Secs. 3.1 and 4.2).
- **Skill-balanced sampling.** To address the imbalance in demonstration frame counts across skills, we increase the sampling share of underrepresented skills during post-training. We limit the expected number of times each frame is sampled (Sec. 4.3).
- **Skill-specific LoRA adapter.** We group skills by the motions they require and train one LoRA adapter for the ac-

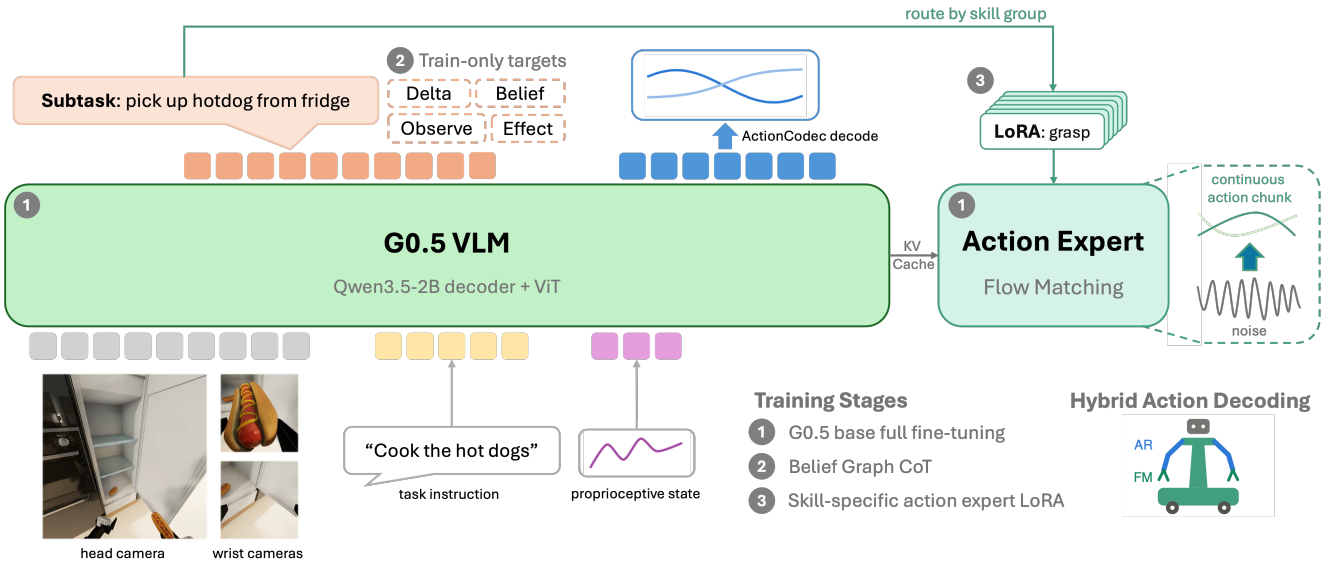


Figure 1. **Overview of our method.** Given three camera views, the task instruction and the proprioceptive state, G0.5 first generates a short subtask (orange tokens) and then the ActionCodec tokens of a 32-step action chunk (blue tokens). During BG-CoT post-training, the same stream is also supervised with belief-graph targets (dashed boxes), which are never generated at test time. The generated subtask selects one of six skill-group LoRA adapters on the flow-matching action expert, which reads the VLM's key-value cache and turns noise into a continuous action chunk. In AR+FM hybrid decoding, the arms come from the ActionCodec tokens and the torso, base and grippers from the action expert. The submitted skill-specific adapter configurations use FM-only decoding. The AR+FM configuration uses the shared action expert. Numbers mark the stage in which each component or target is introduced; the VLM and the action expert are further adapted with LoRA in later stages.

tion expert per skill group. At inference time, the predicted subtask determines which adapter to activate (Sec. 4.4).

- **Hybrid action decoding.** In a separate inference configuration, we decode arm actions from autoregressive action tokens conditioned on the predicted subtask and use the flow-matching action expert to generate continuous actions for the torso, base and grippers (Sec. 4.5).

In full public evaluations across all 100 tasks, BG-CoT post-training with FM decoding reaches a mean Q-score of 0.360, compared with 0.298 for the 700k Stage 1 checkpoint from which it was initialized. Skill-specific LoRA adapters and hybrid decoding do not raise this average further (0.352 and 0.302), but each is the best option on a subset of tasks. For the final submission, we therefore select one of six policy configurations for each task based on its mean Q-score on the task's public instances. Aggregating the same public rollouts for the selected configurations yields a mean Q-score of 0.445 and a full-task success rate of 22.3% across 1,000 episodes (Sec. 5.2).

Pilot evaluations. Closed-loop evaluation took approximately 12 minutes per rollout in our five-task pilot setup. We used these pilots, typically with 25 rollouts per configuration, to guide design choices and devoted most of our evaluation budget to testing candidate configurations across all 100 tasks. We interpret the pilot comparisons as exploratory evidence (Sec. 5.3) and document variants that showed no improvement in the pilot runs (Appendix C).

Code and models. Our training and serving code is available at https://github.com/guthsplan/behavior1k-2026-solution. The submitted Docker image, the model weights, and all 1,000 evaluation JSON files and videos are available at https://huggingface.co/datasets/KWSMooBang/vgi-skt_behavior1k_challenge_2026_submission.

2. Architecture

Base model. We use G0.5 [10] as our base policy (Fig. 1). Its vision-language backbone is a 24-layer Qwen3.5-2B [14] with a hidden size of 2048. Each block of four layers comprises three Gated DeltaNet linear-attention layers [15] followed by one full-attention layer. A 24-layer vision transformer with a hidden size of 1024 and a patch size of 16 encodes each camera view.

G0.5 provides two paths for action generation. The autoregressive (AR) decoder generates reasoning text and discrete action tokens in a single sequence. ActionCodec represents arm and lower-body actions using residual vector quantiza-

tion [17] with two quantization levels and a codebook size of 4,096. It encodes gripper commands as binary tokens. A separate 24-layer action expert with a hidden size of 1024 generates a continuous action chunk through flow matching (FM) [2, 9]. The expert receives the flow time through adaptive layer normalization and attends to the VLM's cached keys and values.

The AR sequence has the following structure, with some formatting tokens omitted for clarity:

$$\underbrace{\langle\text{img}\rangle^3\ \ell\ \mathbf{s}_t}_{\text{prompt}}\ \langle\text{EOC}\rangle\quad\underbrace{\mathbf{c}_t}_{\text{optional CoT}}\quad|\ \text{Action: }\langle\text{EOV}\rangle\quad\underbrace{\mathbf{z}_{t:t+H}}_{\text{action tokens}}\ , \tag{1}$$

where ℓ is the task instruction, $\mathbf{s}_t$ is the proprioceptive embedding, $\mathbf{c}_t$ is the optional chain-of-thought, and $\mathbf{z}_{t:t+H}$ denotes the ActionCodec tokens for the H-step action chunk. During FM training, the expert conditions on the VLM prefix through $\langle\text{EOV}\rangle$, before the discrete action tokens.

Observation and action space. At each policy query, the policy uses the latest RGB frame from the head camera (720×720) and each wrist camera (480×480). We resize all three frames to 256×256. The proprioceptive state comprises seven joint positions and one gripper position per arm, four torso joint positions, and three base velocity components. We zero-pad the seven joint dimensions of each arm to nine, yielding a 27-dimensional representation.

The policy predicts a chunk of $H{=}32$ actions in the same 27-dimensional layout, which we map back to the 23-dimensional R1Pro command space. We express arm and torso targets relative to the joint positions observed at the start of the chunk. We predict base velocities directly and binarize gripper commands at execution. We normalize actions and states using statistics from all training demonstrations, with separate action statistics for each offset within the chunk.

Objective. In the first two training stages, we combine a cross-entropy loss on the supervised AR tokens with a flow-matching loss on the continuous action chunk $\mathbf{a}_{t:t+H}$, abbreviated as $\mathbf{a}$. We denote the input prompt in Eq. (1) as $\mathbf{x}_t = (\langle\text{img}\rangle^3, \ell, \mathbf{s}_t)$ and the supervised token sequence following $\langle\text{EOC}\rangle$ as $\mathbf{y}_t$. In both stages, $\mathbf{y}_t$ contains the action tokens $\mathbf{z}_{t:t+H}$; during post-training, it additionally includes the CoT tokens $\mathbf{c}_t$ when CoT supervision is available. The objective is

$$\begin{aligned}\mathcal{L} = &-\lambda_{\text{CE}}\frac{1}{|\mathbf{y}_t|}\sum_{i=1}^{|\mathbf{y}_t|}\log p_\theta\big(y_{t,i}\mid \mathbf{x}_t, \mathbf{y}_{t,<i}\big)\\ &+\lambda_{\text{FM}}\,\mathbb{E}_{\tau,\boldsymbol{\epsilon}}\Big\|v_\theta\big(\mathbf{a}^\tau, \tau\mid \text{sg}(\text{KV}_t)\big)-(\boldsymbol{\epsilon}-\mathbf{a})\Big\|^2,\end{aligned} \tag{2}$$

where $\mathbf{a}^\tau = (1-\tau)\,\mathbf{a}+\tau\boldsymbol{\epsilon}$ with $\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\mathbf{I})$, and $\tau\in[0,1]$ is drawn from a Beta distribution ($\tau{=}0$: data, $\tau{=}1$: noise). KV_t denotes the cached keys and values of the VLM prefix up to $\langle\text{EOV}\rangle$, and $\text{sg}(\cdot)$ is the stop-gradient operator, so the FM loss updates the action expert without backpropagating through the VLM. We set $\lambda_{\text{CE}} = \lambda_{\text{FM}} = 1$.

ActionCodec omits a gripper's token group from the AR target when the corresponding binary command remains constant throughout the chunk. For example, in the *cook hot dogs* demonstrations, the AR targets contain gripper tokens in only 1.36% of chunks for the left gripper and 2.72% for the right. The FM action expert predicts gripper commands throughout each chunk, which motivates using its outputs for gripper control in the hybrid action decoding configuration (Sec. 4.5).

3. Dataset

BEHAVIOR Challenge 2026. BEHAVIOR-1K [8] defines 1,000 household tasks in fully interactive simulated environments. Each task is specified by initial and goal conditions in BDDL [13]. The 2026 challenge covers 100 tasks across 7 scenes, retaining all 50 tasks from the 2025 edition and adding 50 new ones. The dataset provides 200 teleoperated demonstrations per task, for a total of 20,000 episodes and 210.9 million frames at 30 Hz (approximately 1,950 hours). Each demonstration contains RGB-D video from three cameras, robot proprioception, and actions. The accompanying annotations label skill segments by frame range using 35 skill types, and the corresponding simulator states are available in separate raw HDF5 recordings. At evaluation time, robot observations are restricted to RGB, depth, and proprioception. Privileged simulator states may be used during training.

Skill and task statistics. The distribution of demonstration frames across skills is highly imbalanced (Fig. 2a). Navigation (*move to*) appears in every task and accounts for 38% of all frames, while tool use and in-place interactions (*e.g. turn on switch*, *insert*) account for only 3.1% and 2.3%, respectively (Tab. 1). Of the 35 annotated skills, 26 each account for less than 1% of all frames. We group the skills by their motion and control requirements, including how they use the grippers. We use these groups to define the scope of each skill-specific adapter, training one adapter per group (Sec. 4.4).

Task length also varies widely (Fig. 2(b)). Across tasks, the mean demonstration duration ranges from 72 s to 869 s. The tasks retained from the 2025 edition have a longer mean demonstration duration than the newly introduced tasks (397 s *vs.* 306 s).

3.1. Data Preprocessing

We construct the BG-CoT targets offline on CPUs from the released demonstrations, skill annotations, and recorded simulator states. Target construction does not require replaying

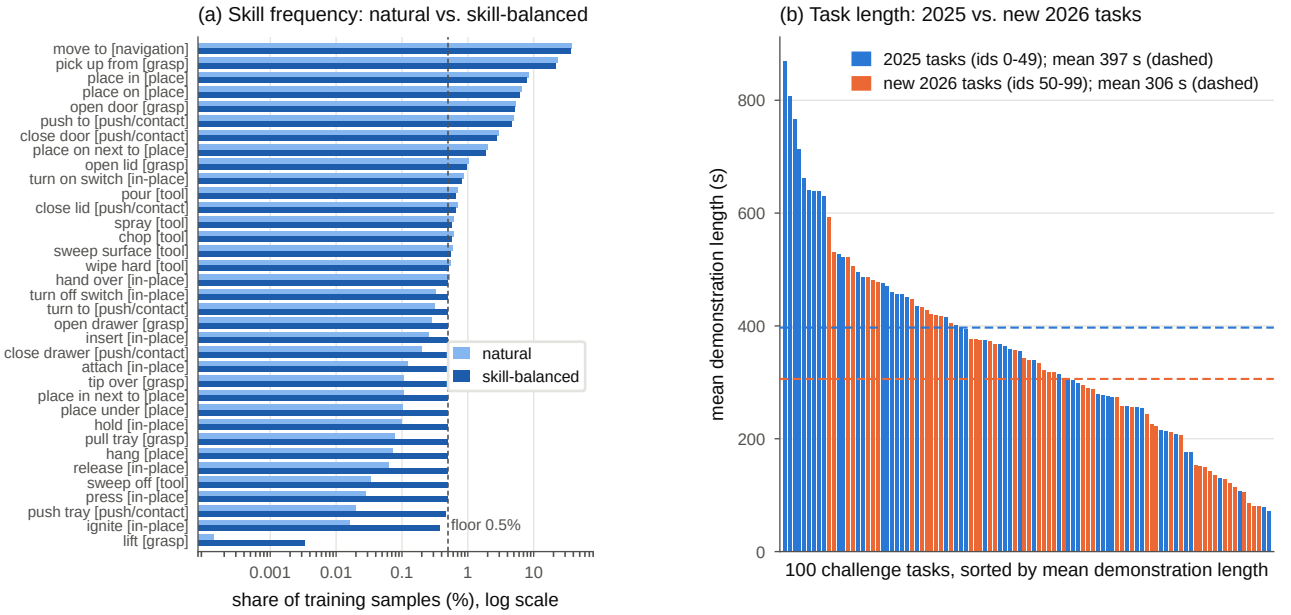


Figure 2. **Dataset statistics** of the 2026 BEHAVIOR Challenge demonstrations. (a) Sampling share of each skill in the training samples under uniform frame sampling (natural) and skill-balanced sampling (Sec. 4.3). The combined share of the 18 skills sampled more frequently rises from 2.2% to 8.4%. For *lift*, which has only 293 demonstration frames, the limit on expected draws per frame restricts its sampling share to about 0.003%. (b) Mean demonstration length per task, estimated by dividing the evaluator's task-specific step limit by 1.5 and converting steps to seconds.

Group	Skills	Frames	Tasks
navigation	move to	38.3%	100
grasp	pick up from, lift, open door/drawer/lid, pull tray, tip over	30.1%	100
place	place in/on/under/on next to/in next to, hang	17.3%	93
push/contact	push to, push tray, close door/lid/drawer, turn to	9.0%	74
tool	chop, sweep surface/off, pour, wipe hard, spray	3.1%	26
in-place	turn on/off switch, press, ignite, insert, hold, release, attach, hand over	2.3%	45

Table 1. **Motion-centric skill groups.** We organize the 35 annotated skills into six groups. For each group, we report its share of demonstration frames and the number of tasks (out of 100) containing at least one of its skills. One configuration also uses an `all` adapter trained on frames from all six groups as a fallback.

the demonstrations in the simulator. At evaluation time, the evaluator provides the task ID along with robot observations. Our policy server uses the ID to look up the corresponding task instruction and passes it to the model along with RGB images and proprioception.

Training dataset and action layout. We build a single LeRobot-format training dataset over all 100 tasks. For every frame, we map the 23-dimensional robot command and the corresponding proprioceptive observation into separate 27-dimensional action and state vectors, grouped by body part (Sec. 2). We precompute normalization statistics from the 100-task dataset using a stride of 100 frames when selecting action-chunk anchors. Stage 2 reuses these statistics. We decode RGB frames from the released videos on the fly. In a five-task pilot test, the checkpoint with the lowest offline validation error (step 54k) succeeded in 5 of 25 closed-loop rollouts, while a later checkpoint (step 120k) succeeded in 7 of 25.

Based on this comparison, we prioritize closed-loop evaluation for checkpoint selection and train the post-training runs (Stages 2 and 3) on all demonstration episodes without an offline validation split.

Subtask labels. We generate a subtask sentence for each annotated skill segment by filling a skill-specific template with the relevant scene objects, *e.g.* `pick up hotdog_208 from fridge_dszchb_0` or `chop log_176 with axe_173`. We use scene instance identifiers to keep object references consistent across subtask, belief, and effect targets. If a skill is interrupted and later resumes, we treat each annotated interval as a separate segment. We assign unannotated frames between consecutive segments to the next skill. In our five-task pilot, subtask

labels covered 99.7% of frames.

Perception labels. G0.5 also supports CoT targets for object bounding boxes and 2D end-effector traces in the head-camera image. We used them only in the five-task pilots, where we encoded the provided annotations with G0.5's `<loc>` tokens (2D traces visible in 73–84% of frames, bounding boxes on 0.8%); the 100-task BG-CoT run uses the subtask and belief-graph targets only.

Belief-graph extraction. We reconstruct simulator states from the raw HDF5 recordings and resolve BDDL object references to scene instances. The scene files provide object categories, scales, and initial poses, while object metadata identifies openable joints. The BDDL problem specifies the initial and goal conditions.

At 10 Hz, we estimate object- and pair-level predicates from recorded simulator states and geometric heuristics. We infer spatial relations (`inside`, `ontop`, `nextto`, `under`, and `touching`) using axis-aligned bounding boxes. We determine `open` from joint positions and `toggled_on`, `cooked`, `frozen`, and `attached` from recorded object states. For robot-object relations, we infer `inhand` from grasp states or, when available, end-effector distance and gripper state, and approximate `reachable` using the horizontal distance from the robot base. We approximate visibility using camera-frustum checks and treat objects inside closed containers as hidden. We calibrate task-specific thresholds using the first 20 demonstrations of each task.

For predicates that cannot be recovered from the recordings, such as particle-based `filled`, we retain their initial values. When an episode terminates successfully, we correct explicitly specified goal literals to their required values. We also track goal progress using BDDL quantifiers, expressed as counts such as `(cooked ?x) 0/2 [hotdog.n.02]`.

To construct the belief graph, we map the 35 annotated skills to templates specifying their preconditions and expected effects. We estimate template statistics from predicate values before and after each annotated skill segment. The belief tracker maintains a truth-value estimate and a confidence score for each predicate. Confidence is initialized at 0.5 and set to 0.97 when a new observation is available. Unobserved predicates retain their previous estimates. When a skill may alter an unobserved spatial relation, we decay its confidence by a factor of 0.8, with a minimum of 0.6. At skill completion, expected effects are provisionally applied to unobserved predicates with a confidence of 0.7. We generate belief-graph targets at 1 Hz and reuse each target across the corresponding 30 Hz training frames.

The pipeline processed all 20,000 episodes in approximately 4.5 hours using 20 parallel CPU processes (96 CPU-hours), generating 7.0 million target rows. For *cook hot dogs*, the extracted predicates achieved 98.5% agreement with an earlier task-specific pipeline. In a validation check of two demonstrations per task, the mean final belief-graph goal progress reached 1.0 for 86 of 100 tasks. The remaining discrepancies involved existential goals over conjunctions, dynamically created objects (*e.g.* egg halves), threshold-sensitive spatial relations (*e.g.* `nextto`), and particle-based predicates.

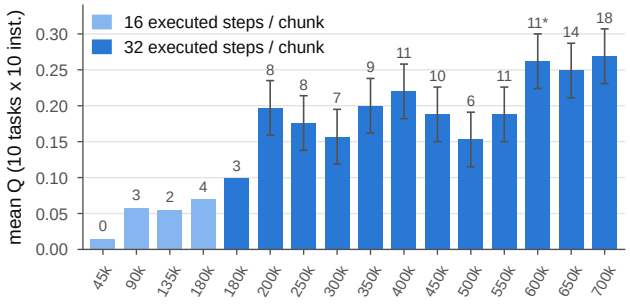


Figure 3. **Stage-1 checkpoints.** Mean Q-score on a fixed subset of 10 tasks with 10 instances per task. Numbers above the bars indicate full successes out of 100 rollouts, except at 600k (*), where 90 rollouts cover nine tasks. Error bars indicate bootstrap standard errors. At the 180k checkpoint, executing all 32 actions in each chunk yields a higher Q-score than executing 16.

4. Method

Our method adapts a shared vision-language backbone in three training stages and evaluates hybrid action decoding as an inference variant (Fig. 1 and Tab. 2). We first fine-tune the full model, then post-train it with BG-CoT supervision, and finally specialize the action expert with skill-specific LoRA adapters. Stages 2 and 3 initialize from checkpoints of the preceding stage.

4.1. G0.5-base: Multi-task Fine-tuning

We fine-tune the official G0.5 checkpoint on all 20,000 demonstrations across the 100 tasks. The input contains RGB observations, an embodiment tag, the task instruction, and an MLP embedding of the proprioceptive state. We train the autoregressive decoder to predict ActionCodec tokens and the flow-matching action expert to predict continuous action chunks (Eq. (2)). This stage uses action supervision alone without chain-of-thought.

We train on eight H200 GPUs with a global batch size of 256 (8×32) using AdamW ($\beta=(0.9, 0.95)$, weight decay 0.03, gradient clipping at 1.0) and BF16 mixed precision. The base learning rate is 8×10^{-5} with 200 warm-up steps and cosine decay.

Figure 3 tracks closed-loop performance during Stage 1. The mean Q-score rises sharply between the 180k and 200k

Stage	Initialization	Trainable modules	Loss	Training data	Steps	Batch	GPUs
1. G0.5-base	Official G0.5	Full model	Action-token CE + FM	All 100 tasks	700k, 800k	256	8× H200
2. BG-CoT	Stage 1 checkpoint	VLM and AE LoRA, selected modules	CoT and action-token CE + FM	All 100 tasks, skill-balanced	180k	128	8× H200
3. Skill-specific adapter	Stage 2 checkpoint	AE LoRA and selected AE modules	FM	Frames from each expert's skill groups	70k–200k	8	1× RTX 4090

Table 2. **Training stages.** AE denotes the action expert. Batch sizes are global. For Stages 1 and 2, Steps lists the checkpoints used in the submission. Stage 1 uses all 20,000 demonstrations. The Stage 2 learning-rate schedule spans 210k steps. Stage 3 step counts are per expert and vary by adapter set (Sec. 4.4).

checkpoints, both evaluated with 32 executed actions per chunk (0.099 to 0.197). This interval spans the training resume, at which the learning rate rose from 2×10^{-6} to 7.1×10^{-5}. From 200k onward, mean Q-scores range from approximately 0.15 to 0.27. In paired bootstrap comparisons over matched task–instance pairs, no later checkpoint shows a statistically significant improvement over an earlier checkpoint in this range. For example, the mean Q-score increases by 0.049 from 400k to 700k (p=0.26). Over the same comparison, the number of full successes increases from 11 to 18. The smoothed training loss decreases from 2.08 at 180k to 1.60 at 750k. Given the uncertainty in the 10-task sweep, we retain both the 700k and 800k checkpoints and evaluate them across all 100 tasks.

4.2. Belief-Graph Chain-of-Thought

Motivation. A policy performing a long household task needs to track which goal conditions already hold, which objects it has manipulated, and what the current subtask should accomplish. A single camera frame may leave this information ambiguous. For example, a robot holding the same object before and after processing it may need to take different actions despite similar observations. To address this problem, we use belief-graph labels extracted from demonstrations to supervise chain-of-thought (CoT) reasoning in the VLM's autoregressive stream [16, 18] without adding an external planner or stage classifier (Sec. 3.1).

CoT targets. We derive four target fields from the belief graph (Tab. 3). **Delta** summarizes goal progress as counts of satisfied conditions relative to the required counts. **Belief** records estimated predicate values for goal-relevant objects, together with their confidence and whether the estimate comes from memory. **Effect** describes the predicate changes expected from the skill assigned to the current frame. **Observe** reports predicate values that are observable in the current frame. We also use G0.5's existing **Subtask** target, which describes the current skill and its associated scene objects.

Field	Meaning	Example
Subtask	current skill with objects	`pick up hotdog_208 from fridge_0`
Delta	satisfied / required goal counts (from BDDL)	`(cooked ?x) 0/2 [hotdog.n.02]`
Belief	object states (memory)	`(open fridge_0) 0.97 mem`
Effect	expected predicate changes for the assigned skill	`(open fridge_0) 1>0`
Observe	currently visible relations	`(open fridge_0) 1`

Table 3. **CoT target fields and examples.** Subtask is an existing CoT target in G0.5. Delta, Belief, Effect, and Observe are auxiliary targets derived from the belief graph. For readability, `fridge_0` abbreviates the scene instance identifier `fridge_dszchb_0`.

Training (Effect builder)
<img>^{3} Embodiment: behavior_r1pro; Task: Cook the hot dogs ... State: <s>; predict symbolic effect of the next skill
<EOC> Effect: (inhand hotdog_208) 0>1, (inside hotdog_208 fridge_dszchb_0) 1>0 |
Action: <EOV> <a_17> <a_903> ... <eos>

Inference (Subtask builder, AR action path)
<img>^{3} Embodiment: behavior_r1pro; Task: Cook the hot dogs ... State: <s>; predict subtask
<EOC> Subtask: pick up hotdog_208 from fridge_dszchb_0 |
Action: <EOV> <a_17> ... <eos>

Figure 4. **Example BG-CoT sequences.** The prompts are shortened, and the action token IDs are illustrative. In the training example, the tokens after <EOC> form the supervised sequence, with the CoT target in orange and the action sequence in blue. At inference, Subtask is the only CoT field generated. The lower example shows how the AR action tokens follow the generated subtask.

Mixed-target training. Generating a long CoT at every step would be slow, we use the belief-graph fields as auxiliary output targets. For each training sample, we select one target builder using fixed weights renormalized over the builders with labels available for that frame (Tab. 7). The selected builder adds a short instruction to the base prompt, such as "predict remaining goal predicates," and supplies the

corresponding CoT target followed by action tokens (Fig. 4). The atomic-task builder supervises action tokens only. A builder may supply one or more CoT fields. At inference time, we request only the **Subtask** field as intermediate reasoning and use the resulting subtask prediction for action generation or skill-specific adapter selection, depending on the policy configuration (Sec. 4.5).

An earlier variant supplied a belief graph derived from simulator labels as part of the training prompt and relied on model estimates to construct the graph at inference time. In a closed-loop pilot, an incorrect observation persisted in the estimated graph until the end of the episode. This failure motivated using the belief graph as supervision during training and generating only the subtask at inference time.

Parameter-efficient training. We apply LoRA [5] with rank 32, α=64, and dropout 0.05 to attention and MLP projections in the VLM and the action expert, including the VLM's linear-attention projections. We also train the action expert's adaptive normalization parameters, input and output projections, time MLP, and final normalization layer, along with the proprioception embedder. The adaptive normalization implementation accesses its weights directly, so we update those weights in full. We freeze the remaining parameters and store them in BF16. We train on eight H200 GPUs with a global batch size of 128 (8×16) and a peak learning rate of 4×10^{-5}, using 200 warm-up steps and a 210k-step cosine schedule. The CoT and skill-adapter configurations in our submission use the Stage 2 checkpoint at 180k steps, reached after about 3.4 days of training.

4.3. Skill-balanced Sampling

The number of demonstration frames varies widely across skills (Fig. 2(a)). For instance, *move to* accounts for approximately 38% of all frames, while other 26 skills each account for less than 1%. Under uniform frame sampling, a skill's share of training samples is proportional to its total frame counts. As a result, skills with few demonstration frames, such as *insert* and *press*, appear infrequently during training. Moreover, the number of frames a skill receives does not depend on how *hard* it is, so short but complex skills are under-trained.

Therefore, we increase the sampling share of underrepresented skills while limiting how often the sampler reuses their frames. Let F_k be the number of frames assigned to skill k, and let R be the planned number of training samples, given by the number of optimization steps times the global batch size. We set the sampling probability of skill k to

$$q_k = \min\Big(\max\left(f,\ \lambda\,\pi_k\right),\ \frac{\kappa F_k}{R}\Big), \qquad \pi_k = \frac{F_k^\alpha}{\sum_j F_j^\alpha}. \tag{3}$$

where f is the target minimum sampling share, κ limits the expected number of draws per frame, and λ is chosen so that $\sum_k q_k = 1$. We draw frames uniformly within each skill, so the expected number of draws per frame is Rq_k/F_k. The limit takes precedence over the target minimum when a skill has too few frames to meet both constraints. The exponent α controls the baseline distribution π_k, from uniform over skills at $\alpha = 0$ to proportional to frame count at $\alpha = 1$. For sampling, we assign unannotated frames between skill segments to the next skill.

In the five-task pilot, $\alpha = 0.75$ yielded a higher mean Q-score than both unbalanced sampling and stronger tempering with $\alpha = 0.5$ (Tab. 6). For post-training on all 100 tasks, we use $\alpha = 1$, $f = 0.5\%$ and $\kappa = 3$ with $R = 210\text{k} \times 128 \approx 26.9$ million samples. The combined share of the 18 skills that receive additional sampling weight rises from 2.2% to 8.4%, while the share of *move to* decreases from 38.3% to 35.9%. For the three rarest skills, the limit on expected draws takes precedence over the target minimum: *push tray* and *ignite* receive 0.47% and 0.38% of the samples, and *lift*, which has only 293 frames, receives about 0.003% (Fig. 2a). A higher target minimum of 1% would allocate approximately 23% of the samples to the 26 skills whose original frame shares are below 1%.

4.4. Skill-specific Action-Expert LoRA

After post-training, a single action expert must still handle a wide range of motions, from driving across a room to grasping a small object or wiping a surface with a tool. Grouping similar tasks is one way to specialize the expert, but each household task typically combines several types of motion. We therefore group skills by their motion and control requirements into six groups: navigation, grasp, place, push/contact, tool use, and in-place interaction (Tab. 1).

Training. For each skill group g, we train a LoRA adapter with rank 32 and α=64 on the attention and MLP projections of the action expert. We also update the expert's input and output projections, time MLP, and final normalization layer. We freeze the VLM, vision encoder, and proprioception embedder and optimize the flow-matching (FM) loss. Each skill-specific adapter trains only on the frames whose annotated skill belongs to g, with the skill-balanced sampling applied within that group. We also train an `all` adapter on every frame as a fallback.

We train two adapter sets from the 180k-step BG-CoT checkpoint. They differ in the context available to the FM action expert.

- **Subtask-conditioned.** The FM action expert attends to the prompt and the generated subtask. This set contains six group adapters and an `all` fallback adapter trained across all skill groups. We train the navigation, grasp, tool-use, and `all` adapter for 150k steps, the place and push/contact adapters for 100k steps, and the in-place adapter for 70k steps.

- **Prompt-conditioned.** The FM action expert uses the prompt context cached before subtask generation. The generated subtask still determines which adapter to activate. This set contains the six group adapters, each trained for 200k steps each, and no `all` adapter.

Routing. At each replanning step, the model generates a subtask. A deterministic parser maps the generated text to a skill label, which determines the skill group and its adapter. During the forward pass, hooks add the active adapter's low-rank updates to the outputs of the adapted linear layers. When the active adapter changes, the router also loads that adapter's parameter values for the modules trained in full.

If the parser cannot identify a skill, the router retains the previously selected group, if one exists. If no group has been selected or the active adapter set has no adapter for the selected group, the router uses the `all` adapter when available and the Stage 2 action expert otherwise. The router requires no learned gating parameters.

4.5. Inference

At each replanning step, the policy decodes a chunk of 32 actions and executes the entire action chunk (Fig. 1). In configurations that use CoT, G0.5 appends the instruction "predict subtask" to the base prompt and generates a `Subtask:` field. When skill-specific adapters are enabled, the policy maps the generated subtask to a skill group and activates the corresponding adapter before decoding the action chunk. At execution, we binarize the gripper commands to ± 1.

We use two action decoding modes. In *FM-only* decoding, the flow-matching head generates actions for all body parts using 10 integration steps. In *AR+FM hybrid* decoding, we decode arm actions from the autoregressive tokens that follow the generated subtask and use the flow-matching head for the torso, base, and grippers. If the AR sequence omits an arm's tokens, we use the FM output for that arm.

This split is motivated by the action codec and our pilot results. Because the AR targets omit a gripper's token group when its binary command stays constant throughout a chunk, the AR decoder rarely emits gripper tokens. The FM action expert predicts gripper commands at every step of the chunk (Sec. 2). In pilot runs that used AR decoding for the lower body, the base remained stationary. On *make microwave popcorn*, pure AR decoding succeeded in 0 of 13 rollouts, while FM-only decoding succeeded in 5 of 18. In a separate five-task pilot, using AR decoding for the arms and FM for the remaining body parts increased the success count from 1 of 25 to 4 of 25 relative to FM-only decoding (Tab. 6).

The number of actions executed before replanning also affected the pilot results. The same policy succeeded in 0 of 24 rollouts when executing 16 actions per chunk and in 3 of 25 when executing all 32. In the Stage 1 sweep, the 180k checkpoint's mean Q-score increased from 0.070 to 0.099 when we executed 32 rather than 16 actions per chunk (Fig. 3).

The implementation supports combining skill-specific adapters with hybrid decoding by using the selected adapter for the FM outputs. Within the challenge evaluation budget, we evaluated skill-specific adapters and hybrid decoding in separate configurations (Sec. 4.6).

4.6. Submission Policy

No single configuration achieved the highest mean Q-score on every task in our public evaluations. Our submission therefore uses a fixed mapping from tasks to six policy configurations (Tab. 5). These comprise two action-only G0.5-base checkpoints, two BG-CoT configurations using FM-only and AR+FM hybrid action decoding, respectively, and two BG-CoT configurations using FM with the skill-specific adapter sets described in Sec. 4.4. All six share the same action codec and normalization statistics. Together, they use three base checkpoints (Stage 1 at 700k and 800k steps and Stage 2 at 180k steps) and 13 skill-specific adapters.

For each task, we compare configurations with results on all 10 public instances and select the one with the highest mean Q-score. We break ties first by the number of full successes and then by consulting results on similar tasks, where similarity is based on skills, goal predicates, and objects. The resulting mapping assigns the same configuration to every instance of a task.

We package all six submission configurations in a single Docker image and verify that the image can serve them on a single 24 GB GPU. In a serving test that switched among configurations, peak reserved GPU memory was approximately 17 GiB. The policy server reads the task ID from the first observation of each episode and uses the fixed mapping to select the configuration.

We compute the reported public score from the same rollouts used to select the task-to-configuration mapping. The score therefore summarizes performance on the public instances used for selection. An independent estimate of generalization requires evaluation on held-out instances (Sec. 5.2).

5. Experiments

5.1. Implementation and Evaluation Protocol

We train our models in PyTorch with BF16 autocast. Stage 1 fine-tunes the full model on 8 H200 GPUs with a global batch size of 256 (Sec. 4.1). Stage 2 applies LoRA to both the VLM and the action expert and trains with the combined autoregressive and flow-matching objective on 8 H200 GPUs at a global batch size of 128 (Sec. 4.2). Stage 3 freezes the VLM and trains the skill-specific adapters of the action expert using only the flow-matching loss, with training runs ranging from 70k to 200k steps (Sec. 4.4).

Task subset	#Tasks	Demo len.	SR	Q-score
2025 tasks (ids 0–49)	50	397 s	0.168	0.395
New 2026 tasks (ids 50–99)	50	306 s	0.278	0.495
All	100	352 s	0.223	0.445

Table 4. **Public evaluation results by task origin.** We report the full-task success rate (SR) and mean Q-score of the task-routed submission on public instances 301–310, grouped by whether a task was retained from the 2025 challenge or introduced in 2026. Demo len. denotes the mean demonstration length for each subset.

Evaluation. We use the official OmniGibson evaluator with the default R1Pro robot configuration and the `RGBDFullResWrapper`. Our public evaluations use BEHAVIOR-1K v3.9.2–v3.9.3-post1 depending on configuration. For each candidate configuration, we schedule one rollout on each of the 10 public instances (301–310) for each of the 100 tasks. We obtain 5,985 rollout records across the six candidates and aggregate 1,000 of these using the fixed task-to-configuration mapping described in Sec. 4.6. We use the evaluator's default episode limit of $1.5\times$ the mean human demonstration length for each task.

We report the **Q-score**, the fraction of BDDL goal conditions satisfied at the end of an episode. We average Q-scores over instances within each task and then over tasks. This is the challenge's primary ranking metric. We report the **full-task success rate** (SR) as the fraction of episodes marked successful by the evaluator. We also report **normalized time** (NT), the ratio of the task's mean demonstration duration to the episode's simulated duration.

5.2. Main Results

Table 5 summarizes our public evaluations. We first compare the six configurations individually on all 100 tasks and then report the task-routed submission.

Single configurations. The *All 100 tasks* columns of Tab. 5 report scores over 1,000 public rollouts per configuration, counting missing rollouts as failures with zero Q-score. BG-CoT post-training with FM decoding achieves the highest mean Q-score (0.360), compared with 0.298 for the 700k G0.5-base checkpoint from which it was initialized and 0.260 for the 800k checkpoint. This gain also includes 180k additional training steps with the skill-balanced sampler. Subtask-conditioned skill-specific adapters reach a similar Q-score (0.352) with the highest success rate (16.1%), whereas prompt-conditioned adapters (0.309) and AR+FM hybrid action decoding (0.302) are lower on average.

Task-routed submission. The best configuration differs across tasks (*Tasks assigned in submission* columns of Tab. 5). Of the 100 tasks, 81 are assigned to BG-CoT configurations, including 35 assigned to configurations with skill-specific adapter configuration. Routing each task to the configuration with the highest mean public Q-score raises the aggregate public Q-score from 0.360 for the best single configuration to 0.445 and the full-task success rate from 15.1% to 22.3% (223 of 1,000 episodes). Successful rollouts have a mean normalized time of 0.93, and all 777 unsuccessful rollouts reach the evaluator's step limit. Because routing and scoring use the same rollouts, part of this gain is a selection effect.

Performance varies across tasks (Fig. 5). The policy succeeds at least once on 69 tasks and in at least 5 of 10 rollouts on 17 tasks. It succeeds in 9 of 10 rollouts on both *moving boxes to storage* and *putting dirty dishes in sink*, with a mean Q-score of 0.95 on each task. The policy also earns substantial partial credit on some tasks without a full success. For example, it achieves a mean Q-score of 0.81 on *setup a bar for a cocktail party*, with no full successes across 10 rollouts. Only three tasks have a mean Q-score of zero.

Table 4 reports results by task origin. The 50 tasks retained from the 2025 challenge have a lower mean Q-score and full-task success rate (0.395 and 16.8%) than the 50 new tasks (0.495 and 27.8%). Demonstrations for the retained tasks are about 30% longer on average. Across all 100 tasks, mean demonstration length is negatively correlated with full-task success rate (Spearman $\rho{=}-0.27$, $p{=}0.006$) and shows little rank correlation with mean Q-score ($\rho{=}0.02$; Fig. 6). These associations describe how performance varies with task length in our public evaluation. The two subsets also differ in task composition.

5.3. Component Analysis

Closed-loop evaluation is slow (about 12 minutes per rollout), so we first examined individual design choices on five-task pilots (Tab. 6) and then evaluated the resulting configurations on all 100 tasks (Tab. 5). We treat the pilots as exploratory and note where the full evaluation disagrees with them.

BG-CoT. The configuration trained with belief-graph supervision achieved a mean Q-score of 0.157, compared with approximately 0.10 for the No-CoT configuration. At inference time, the BG-CoT configuration generates only the subtask as its CoT output. The belief-graph fields provide additional supervision during training. On all 100 tasks, BG-CoT post-training with FM decoding reaches 0.360, compared with 0.298 for its 700k initialization (Tab. 5); this comparison also includes the skill-balanced sampling and 180k additional training steps.

Skill-balanced sampling. Mild tempering ($\alpha{=}0.75$) yielded the highest mean Q-score in the five-task pilot (0.240), compared with 0.180 for unbalanced sampling and

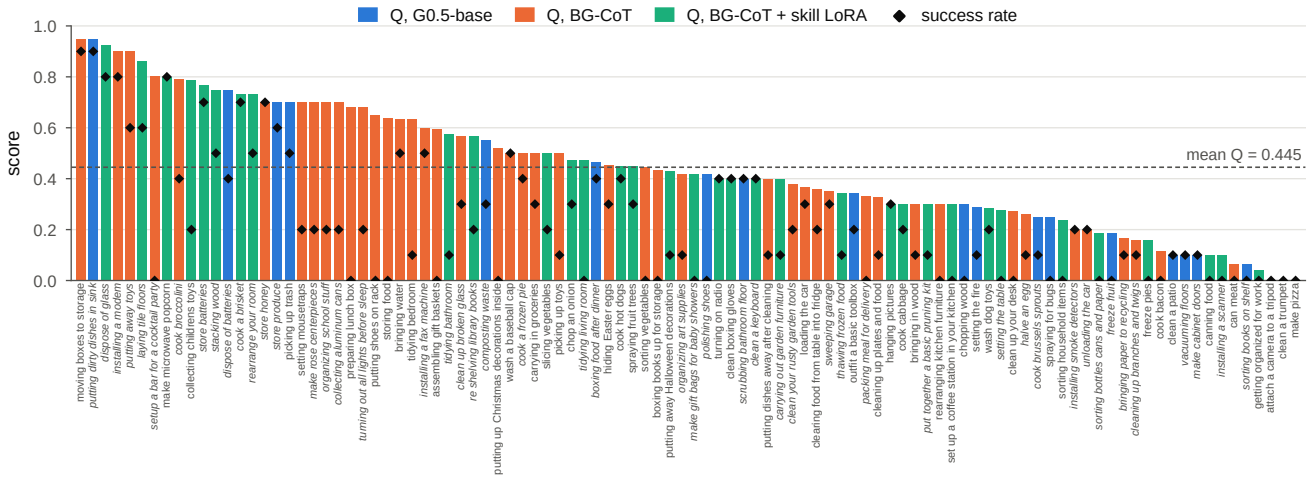


Figure 5. **Per-task results on public instances 301–310.** Tasks are ordered by decreasing mean Q-score. Bar height shows the mean Q-score over the 10 instances for each task, and bar color identifies the policy family assigned to that task. Black diamonds indicate full-task success rates, and the dashed line marks the mean Q-score across all 100 tasks (0.445). Italicized task names denote tasks introduced in 2026. Per-task values are reported in Tabs. 10 and 11.

						All 100 tasks		Tasks assigned in submission		
Configuration	Base ckpt.	CoT	Weights	Decoding	Skill-specific adapters	SR	Q	#Tasks	SR	Q
G0.5-base @700k	Stage 1 @700k	✗	fp32	FM	–	0.132	0.298	9	0.289	0.435
G0.5-base @800k	Stage 1 @800k	✗	bf16	FM*	–	0.108	0.260	10	0.160	0.300
BG-CoT, FM-only	Stage 2 @180k	✓	bf16	FM	–	0.151	**0.360**	36	0.181	0.495
BG-CoT, AR+FM hybrid	Stage 2 @180k	✓	fp32	AR arms + FM	–	0.136	0.302	10	0.300	0.454
BG-CoT + skill LoRA (a)	Stage 2 @180k	✓	fp32	FM	7, subtask-cond.	**0.161**	0.352	22	0.250	0.440
BG-CoT + skill LoRA (b)	Stage 2 @180k	✓	fp32	FM	6, prompt-cond.	0.113	0.309	13	0.238	0.427
Task-routed submission	per-task best of the six configurations on instances 301–310					0.223	0.445	100	0.223	0.445

Table 5. **Public evaluation results for the submission configurations.** The *All 100 tasks* columns report results over the 1,000 scheduled rollouts per configuration, with missing rollouts counted as zero Q-score and unsuccessful. The 700k base and FM-only BG-CoT runs produced 986 and 999 rollout records, respectively. The *Tasks assigned in submission* columns report results only on the tasks assigned to each configuration by public Q-score, so rows cover different task subsets. Weights indicates the storage precision of the model weights. Inference uses BF16 autocast throughout. (a) and (b) denote the adapter sets described in Sec. 4.4. Simulator: BEHAVIOR-1K v3.9.2 for the G0.5-base rows, v3.9.3-post1 for the two BG-CoT rows without adapters, and v3.9.3 for the skill-adapter rows. Bold marks the best single configuration in each of the *All 100 tasks* columns. *When the AR decode omits a gripper or lower-body token group, that part's FM output is replaced by a hold command (previous gripper command, current torso position, zero base velocity).

0.170 for stronger tempering (α=0.5). For post-training on all 100 tasks, we use frame-proportional sampling weights (α=1) as the baseline and increase the share of underrepresented skills subject to the limit on expected draws per frame (Sec. 4.3).

AR+FM hybrid. Decoding the arms with AR and the remaining body parts with FM yielded 4 successes in 25 pilot rollouts, compared with 1 in 25 for FM-only decoding; the additional successes occurred on *installing a modem* (3/5 *vs*. 1/5) and *make microwave popcorn* (1/5 *vs*. 0/5). On all 100 tasks, however, the hybrid configuration has a lower mean Q-score than FM-only decoding of the same checkpoint (0.302 *vs*. 0.360; SR 13.6% *vs*. 15.1%), although it was the best configuration on 10 tasks. Its benefit is therefore task-dependent and the two runs also differ in weight precision (FP32 *vs*. BF16).

Skill-specific adapters. On all 100 tasks, subtask-conditioned adapters reach a mean Q-score of 0.352 and a success rate of 16.1%, compared with 0.360 and 15.1% for the shared action expert of the same BG-CoT checkpoint, while prompt-conditioned adapters are lower (0.309). Adapter configurations were nevertheless the best choice on 35 tasks.

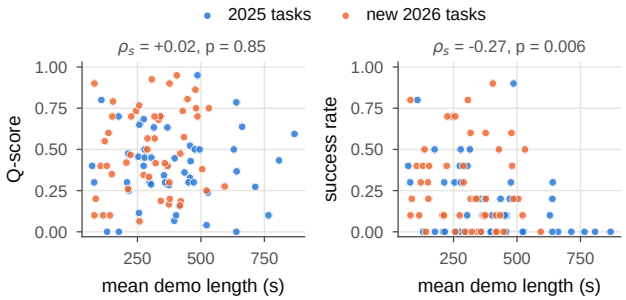


Figure 6. **Task length *vs*. Q-score and success rate.** Each point represents one task evaluated on 10 public instances using its assigned policy configuration. Mean demonstration duration is negatively associated with full-task success rate and shows little rank correlation with mean Q-score.

Component	Setting	Success	Q
CoT targets†	No-CoT	–	≈0.10
	BG-CoT	–	0.157
Skill balancing‡	none	4/25	0.180
	α=0.5	3/25	0.170
	α=0.75	5/25	0.240
Decoding‡	FM only	1/25	0.080
	AR+FM hybrid	4/25	0.180
Executed steps§	16 of 32	0/24	–
	32 of 32	3/25	–
Checkpoint¶	best offline val. L1 (54k)	5/25	–
	later checkpoint (120k)	7/25	–

Table 6. **Component analysis on five-task pilot sets.** Success counts are reported over the available rollouts. Dashes indicate unreported metrics. † The CoT comparison uses tasks 4, 28, 40, 44, and 45. ‡ The skill-balancing and decoding comparisons use tasks 0, 34, 40, 69, and 77, with five instances per task. The "none" and "AR+FM hybrid" rows come from the same run. § The chunk-execution comparison uses FM decoding on the first pilot set. ¶ The checkpoint comparison uses a pilot with an offline validation split.

Chunk execution and checkpoint selection. Executing the whole 32-step chunk before replanning was better than executing half of it in both comparisons we ran: 3 of 25 *vs*. 0 of 24 successes in the pilot, and Q 0.099 *vs*. 0.070 at the same 180k checkpoint in the Stage 1 sweep (Fig. 3). The checkpoint with the lowest offline validation error also performed worse in closed loop than a later one (5/25 *vs*. 7/25; Tab. 6), so for the post-training runs we use all demonstration episodes and assess candidate checkpoints through closed-loop evaluation.

Failure modes. In a qualitative review of failed rollouts, we observed repeated grasp and placement attempts, selection of a hand poorly positioned for the target object, and collisions from which the robot failed to recover. We observed these behaviors in policies with and without CoT. We also observed arm and torso posture drift, which sometimes tilted the head camera toward the ceiling or floor and shifted the view away from those seen in training. We report additional inference variants and their pilot results in Appendix C.

6. Conclusion

We presented Team VGI-SKT's solution to the 2026 BEHAVIOR Challenge. Starting from G0.5, we fine-tuned the model on demonstrations from all 100 tasks, post-trained it with belief-graph chain-of-thought supervision and skill-balanced sampling, and trained skill-specific LoRA adapters for the action expert. We also evaluated hybrid decoding, which combines autoregressive arm actions with flow-matching outputs for the torso, base, and grippers. Our task-routed submission achieved a mean Q-score of 0.445 and a full-task success rate of 22.3% across 1,000 public rollouts, the same rollouts used to select its configurations.

Belief-graph supervision teaches the model to predict task-state information during training. At inference time, the policy generates a short subtask description and, when skill-specific adapters are enabled, uses it to select corresponding adapter. Hybrid action decoding provides a separate way to adapt action generation to the robot's body parts, using autoregressive tokens for the arms and flow matching for the torso, base, and grippers. Both decoding modes execute the full action chunk before replanning.

Three lessons stand out. First, belief-graph supervision, *i.e*. symbolic task-state targets used only as auxiliary outputs, improved the pilot policy (Q 0.157 *vs*. about 0.10) without maintaining a belief graph at test time. On all 100 tasks, BG-CoT post-training, which also adds the skill-balanced sampling and 180k further training steps, raised the Q-score from 0.298 to 0.360 over its initialization. Second, routing skill-specific adapters by the predicted subtask adds no parameters and works reliably, but at the scale of 100 tasks the skill-specific adapters did not raise the average Q-score over a shared action expert; they were the best choice on 35 tasks. Third, inference-time choices matter: executing the full 32-step chunk consistently helped, whereas AR+FM hybrid action decoding helped on some tasks but not on average.

Limitations and future work. Our component comparisons use small pilot sets. Controlled comparisons across all 100 tasks are needed to quantify the contribution of each component. We use the same public rollouts to select configurations and report the routed score, so the public improvement over a single configuration reflects performance on the instances used for selection. Our full-run comparisons also

mix weight precisions. Longer tasks were completed less often in our public evaluations. The policy also lacks an explicit mechanism to detect and retry a failed subgoal. Future work includes evaluating skill-specific adapters and hybrid decoding together, testing whether predicted belief states can support subgoal recovery, and using policy-generated rollouts for further training through rejection-sampling fine-tuning [1] or reinforcement learning.

Acknowledgements. We thank the organizers of the BEHAVIOR Challenge for providing the benchmark and demonstrations and for supporting the evaluation. We also thank the authors of G0.5 for releasing their model. We gratefully acknowledge SK Telecom for providing computing resources for this work. We would like to express our special thanks to Chunfei Ma, Seungjun Lee, and Joonhyang Choi, who led the experiments on SK Telecom's computing infrastructure.

7. Authors

Authors: Woosung Kim*, Hyungu Kim*, Hoyong Kim*, Chunfei Ma, Seungjun Lee, Joonhyang Choi, Haebeom Jung, Hyunjoon Lee, Sunghyun Choi, Daeun Lee, Eunsung Cha, Seokmin Song, Junghwan Park, Jaesik Park

References

[1] Junjie Bai, Yu-Wei Chao, Qizhi Chen, Jinwei Gu, Moo Jin Kim, Zhaoshuo Li, Xuan Li, Tsung-Yi Lin, Ming-Yu Liu, Nic Ma, Kaichun Mo, Delin Qu, Shangkun Sun, Hongchi Xia, Fangyin Wei, and Xiaohui Zeng. Openpi comet: Competition solution for 2025 BEHAVIOR challenge. *arXiv preprint arXiv:2512.10071*, 2025. 1, 12

[2] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. 1, 3

[3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. RT-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.

[4] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023. 1

[5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. 7

[6] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024. 1

[7] Ilia Larchenko, Gleb Zarin, and Akash Karnatak. Task adaptation of vision-language-action model: 1st place solution for the 2025 BEHAVIOR challenge. *arXiv preprint arXiv:2512.06951*, 2025. 1

[8] Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Wensi Ai, Benjamin Martinez, et al. BEHAVIOR-1K: A human-centered, embodied AI benchmark with 1,000 everyday activities and realistic simulation. *arXiv preprint arXiv:2403.09227*, 2024. 1, 3

[9] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023. 3

[10] Yicheng Liu, Zibin Dong, Baijun Ye, Tianyuan Yuan, Tao Jiang, Anqi Yang, Shicheng Cao, Haonan Liu, Yue Sun, Zihan Guo, Xiao Liu, Dong Ke, Changxun Pan, Chenru Wu, Tailai Cheng, Xiaoshu Ren, Xinlei Zhang, Jianning Cui, Zijie Zhao, Haoyu Zhang, Kaiming Xu, Haodong Yang, Bowen Zhang, Jiahui Niu, Shaoting Zhu, Shiduo Zhang, and Hang Zhao. G0.5: One autoregressive stream for robot reasoning and action. arXiv preprint arXiv:2608.11739, 2026. 1, 2

[11] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Charles Xu, Jianlan Luo, et al. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems (RSS)*, 2024. 1

[12] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 1

[13] Sanjana Srivastava, Chengshu Li, Michael Lingelbach, Roberto Martín-Martín, Fei Xia, Kent Elliott Vainio, Zheng Lian, Cem Gokmen, Shyamal Buch, Karen Liu, et al. BEHAVIOR: Benchmark for everyday household activities in virtual, interactive, and ecological environments. In *Conference on Robot Learning (CoRL)*, 2022. 3

[14] Qwen Team. Qwen3.5: Towards native multimodal agents. https://www.alibabacloud.com/blog/qwen3-5-towards-native-multimodal-agents_602894, 2026. 2

[15] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2024. 2

[16] Michał Zawalski, William Chen, Karl Pertsch, Oier Mees, Chelsea Finn, and Sergey Levine. Robotic control via embodied chain-of-thought reasoning. *arXiv preprint arXiv:2407.08693*, 2024. 6

[17] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. SoundStream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30:495–507, 2021. 3

[18] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han,

*These authors contributed equally to this work.

Chelsea Finn, et al. CoT-VLA: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1702–1713, 2025. 6

[19] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems (RSS)*, 2023. 1

VGI-SKT: Competition Solution for the 2026 BEHAVIOR Challenge

Supplementary Material

A. BG-CoT Target Builders

Table 7 lists the target builders of Stage 2. For each sample, one builder is drawn by weight among the builders whose labels exist for the frame. Multi-field builders apply only when all of their fields are present, so the single-field builders are always kept as candidates. The longest sequence (Full belief) uses about 420 tokens, well below the 1,200-token limit of a chunk.

Builder	Weight	Task prompt / supervised fields
Atomic task (no CoT)	1.0	subtask sentence placed in `Task:`, actions only
Subtask	1.0	"predict subtask"
Delta	0.5	"predict remaining goal predicates"
Belief update	0.5	"predict updated belief graph"
Observe	0.5	"predict visible predicates"
Effect	0.5	"predict symbolic effect of the next skill"
Full belief	0.2	Observe | Belief | Delta | Effect
Observe + Update	0.4	Observe | Belief
Subtask + Effect	0.4	Subtask | Effect

Table 7. **Target builders** for BG-CoT training. All builders end with the action tokens. At inference the Subtask builder is used.

B. Hyperparameters

Table 8 summarizes the hyperparameters of the three training stages and of inference.

Model and data	
Cameras / resolution	head + 2 wrists, 256×256, 1 frame
State / action layout	27-D in 5 groups, chunk $H=32$ at 30 Hz
ActionCodec	RVQ, 2 levels, 4096 codes, block size 8
FM time sampling / steps	Beta / 10 integration steps
Stage 1	
Optimizer	AdamW, lr 8×10^{-5}, wd 0.03
Schedule	200 warm-up steps, cosine to 0
Batch / steps	256 (8×32) / 823,894 (1 epoch)
FM samples per VLM pass	4
Stage 2 (BG-CoT)	
LoRA	$r=32$, $\alpha=64$, dropout 0.05, VLM + AE
Full-trained modules	AE adaLN, AE IO/time MLP/norm, proprio emb.
Optimizer / schedule	AdamW, lr 4×10^{-5}, cosine over 210k
Batch / steps used	128 (8×16) / 180k
Sampler	$\alpha=1$, floor $f=0.5\%$, cap $\kappa=3$, $R=26.88$M
Stage 3 (skill LoRA)	
LoRA	$r=32$, $\alpha=64$, AE only
Full-trained modules	AE IO, time MLP, final norm
Loss / lr	FM only / 4×10^{-5}
Steps	set (a): 70k–150k; set (b): 200k
Inference	
Replanning	every 32 steps (full chunk)
Gripper	binarized to ±1
Routing	switch on every new parsed group

Table 8. **Hyperparameters** of the final recipe. AE: action expert; IO: input/output projections.

C. Inference Options That Did Not Help

Before settling on the decoding scheme of Sec. 4.5, we tried several inference-time options on the pilot tests, mostly on *make microwave popcorn*, where the FM-only baseline succeeded in 5 of 18 rollouts (Tab. 9). None of them improved over the baseline, and they are disabled in all submitted configurations. The same pilots showed that failures were dominated by repeated precise-manipulation attempts, hand assignment and missing recovery after collisions, independent of the CoT, which points to data (recovery and retry demonstrations) rather than inference tricks as the next lever.

Option	Popcorn success
Baseline: CoT serving, FM action expert, 32-step chunks	5/18
Subtask hysteresis / no-regression filter on the CoT	0/11
ACT-style temporal ensembling [19]	0/5
Inpainting the first 4 steps from the previous chunk (28 executed)	4/20
Proprioception bias correction (command–joint offset of the demos)	3/20
AR head for all body parts	0/13

Table 9. **Inference options** evaluated on *make microwave popcorn*. Serving without CoT was on par with CoT serving on the 5-task set (2/25 *vs*. 3/25).

D. Per-task Results

Tables 10 and 11 list the results of our submission for every task on public instances 301–310 (one rollout each). Configuration names follow Tab. 5: Base-700k / Base-800k = G0.5-base checkpoints; CoT-FM / CoT-Hybrid = BG-CoT with FM-only / AR+FM hybrid action decoding; SkLoRA-a / SkLoRA-b = BG-CoT + skill-specific LoRA (adapter sets a / b). NT = mean normalized time; T = mean simulated seconds per episode; T_{demo} = mean demonstration length.

ID	Task	Config.	SR	Q	NT	T (s)	T_{demo} (s)
0	turning_on_radio	SkLoRA-b	4/10	0.400	0.86	90	72
1	picking_up_trash	Base-800k	5/10	0.700	0.84	224	176
2	putting_away_Halloween_decorations	SkLoRA-a	1/10	0.429	0.69	671	460
3	cleaning_up_plates_and_food	CoT-FM	1/10	0.329	0.69	668	457
4	can_meat	CoT-FM	0/10	0.067	0.67	592	395
5	setting_mousetraps	CoT-FM	2/10	0.700	0.71	487	340
6	hiding_Easter_eggs	CoT-Hybrid	3/10	0.456	0.75	350	254
7	picking_up_toys	CoT-FM	1/10	0.500	0.68	929	630
8	rearranging_kitchen_furniture	CoT-FM	0/10	0.300	0.67	447	298
9	putting_up_Christmas_decorations_inside	CoT-FM	0/10	0.522	0.67	686	457
10	set_up_a_coffee_station_in_your_kitchen	SkLoRA-b	0/10	0.300	0.67	313	209
11	putting_dishes_away_after_cleaning	CoT-FM	1/10	0.400	0.68	541	365
12	preparing_lunch_box	CoT-FM	0/10	0.683	0.67	412	275
13	loading_the_car	CoT-Hybrid	3/10	0.367	0.74	886	641
14	carrying_in_groceries	CoT-FM	3/10	0.500	0.72	669	476
15	bringing_in_wood	CoT-Hybrid	1/10	0.300	0.69	659	451
16	moving_boxes_to_storage	CoT-FM	9/10	0.950	1.01	491	487
17	bringing_water	CoT-FM	5/10	0.633	0.80	413	315
18	tidying_bedroom	CoT-FM	1/10	0.633	0.69	539	368
19	outfit_a_basic_toolbox	Base-700k	2/10	0.343	0.71	510	355
20	sorting_vegetables	CoT-FM	0/10	0.446	0.67	595	397
21	collecting_childrens_toys	SkLoRA-a	2/10	0.786	0.73	897	640
22	putting_shoes_on_rack	CoT-FM	0/10	0.650	0.67	386	258
23	boxing_books_up_for_storage	CoT-FM	0/10	0.433	0.67	1211	808
24	storing_food	CoT-FM	0/10	0.637	0.67	993	662
25	clearing_food_from_table_into_fridge	CoT-FM	2/10	0.360	0.69	634	436
26	assembling_gift_baskets	CoT-FM	0/10	0.594	0.67	1303	869
27	sorting_household_items	SkLoRA-b	0/10	0.237	0.67	790	527
28	getting_organized_for_work	SkLoRA-a	0/10	0.040	0.67	784	522
29	clean_up_your_desk	CoT-FM	0/10	0.273	0.67	1071	714
30	setting_the_fire	Base-700k	1/10	0.287	0.67	454	304
31	clean_boxing_gloves	SkLoRA-a	4/10	0.400	0.69	399	275
32	wash_a_baseball_cap	CoT-Hybrid	5/10	0.500	0.73	386	278
33	wash_dog_toys	SkLoRA-a	2/10	0.283	0.68	553	374
34	hanging_pictures	SkLoRA-a	3/10	0.300	0.86	104	80
35	attach_a_camera_to_a_tripod	CoT-FM	0/10	0.000	0.67	196	130
36	clean_a_patio	Base-800k	1/10	0.100	0.71	578	402
37	clean_a_trumpet	Base-700k	0/10	0.000	0.67	265	177
38	spraying_for_bugs	Base-800k	0/10	0.250	0.67	324	216
39	spraying_fruit_trees	SkLoRA-b	3/10	0.450	0.76	380	278
40	make_microwave_popcorn	SkLoRA-a	8/10	0.800	0.87	128	108
41	cook_cabbage	SkLoRA-a	2/10	0.300	0.70	678	471
42	chop_an_onion	SkLoRA-b	3/10	0.475	0.67	316	213
43	slicing_vegetables	SkLoRA-a	2/10	0.500	0.68	728	495
44	chopping_wood	Base-800k	0/10	0.300	0.67	538	358
45	cook_hot_dogs	SkLoRA-a	4/10	0.450	0.75	414	305
46	cook_bacon	CoT-FM	0/10	0.114	0.67	384	256
47	freeze_pies	SkLoRA-b	0/10	0.157	0.67	623	415
48	canning_food	SkLoRA-a	0/10	0.100	0.67	1149	766
49	make_pizza	SkLoRA-a	0/10	0.000	0.67	959	640
	mean over 50 tasks		84/500	0.395	0.71		

Table 10. **Per-task results, tasks 0–49** (2025 challenge tasks).

ID	Task	Config.	SR	Q	NT	T (s)	T_{demo} (s)
50	freeze_fruit	Base-800k	0/10	0.186	0.67	631	421
51	cook_a_brisket	SkLoRA-b	7/10	0.733	0.78	319	244
52	sorting_bottles_cans_and_paper	SkLoRA-b	0/10	0.188	0.67	513	342
53	tidying_living_room	SkLoRA-a	0/10	0.475	0.67	628	419
54	putting_away_toys	CoT-FM	6/10	0.900	0.72	528	377
55	re_shelving_library_books	SkLoRA-a	2/10	0.567	0.68	468	318
56	make_rose_centerpieces	CoT-FM	2/10	0.700	0.72	214	151
57	sweeping_garage	CoT-Hybrid	3/10	0.350	0.72	211	149
58	stacking_wood	SkLoRA-a	5/10	0.750	0.71	756	531
59	organizing_art_supplies	CoT-FM	1/10	0.420	0.69	301	206
60	scrubbing_bathroom_floor	Base-700k	4/10	0.400	0.78	140	105
61	bringing_paper_to_recycling	CoT-FM	1/10	0.167	0.69	544	374
62	halve_an_egg	CoT-FM	1/10	0.260	0.68	314	213
63	installing_smoke_detectors	CoT-FM	2/10	0.200	0.72	122	86
64	setting_the_table	SkLoRA-b	0/10	0.275	0.67	890	594
65	unloading_the_car	CoT-FM	2/10	0.200	0.72	533	377
66	turning_out_all_lights_before_sleep	CoT-FM	2/10	0.680	0.71	477	334
67	boxing_food_after_dinner	Base-800k	4/10	0.467	0.78	299	223
68	cleaning_up_branches_and_twigs	CoT-Hybrid	1/10	0.160	0.67	620	418
69	vacuuming_floors	Base-800k	1/10	0.100	0.68	119	80
70	thawing_frozen_food	SkLoRA-a	1/10	0.344	0.69	402	274
71	clean_your_rusty_garden_tools	CoT-Hybrid	2/10	0.380	0.70	729	505
72	cook_a_frozen_pie	CoT-FM	4/10	0.500	0.72	407	289
73	organizing_school_stuff	CoT-FM	2/10	0.700	0.74	679	486
74	carrying_out_garden_furniture	SkLoRA-a	1/10	0.400	0.68	542	369
75	put_together_a_basic_pruning_kit	SkLoRA-a	1/10	0.300	0.68	556	375
76	dispose_of_glass	SkLoRA-b	8/10	0.925	0.88	360	306
77	installing_a_modem	CoT-Hybrid	8/10	0.900	1.05	81	80
78	make_cabinet_doors	Base-800k	1/10	0.100	0.67	170	115
79	polishing_shoes	Base-700k	0/10	0.417	0.67	537	358
80	clean_up_broken_glass	CoT-FM	3/10	0.567	0.77	394	290
81	packing_meal_for_delivery	CoT-Hybrid	0/10	0.333	0.67	443	295
82	store_batteries	SkLoRA-a	7/10	0.767	0.87	309	257
83	store_honey	CoT-FM	7/10	0.700	1.24	220	226
84	tidying_bathroom	SkLoRA-b	1/10	0.575	0.69	633	433
85	putting_dirty_dishes_in_sink	Base-700k	9/10	0.950	0.98	422	405
86	make_gift_bags_for_baby_showers	SkLoRA-a	0/10	0.417	0.67	483	322
87	collecting_aluminum_cans	CoT-FM	2/10	0.700	0.68	499	340
88	rearrange_your_room	SkLoRA-b	5/10	0.733	0.72	603	428
89	installing_a_fax_machine	CoT-FM	5/10	0.600	0.73	188	136
90	composting_waste	Base-800k	3/10	0.550	0.71	174	121
91	store_produce	Base-700k	6/10	0.700	0.85	395	317
92	installing_a_scanner	SkLoRA-b	0/10	0.100	0.67	213	142
93	clean_a_keyboard	SkLoRA-a	4/10	0.400	0.77	173	129
94	dispose_of_batteries	Base-700k	4/10	0.750	0.78	635	481
95	cook_brussels_sprouts	Base-800k	1/10	0.250	0.67	782	522
96	cook_broccolini	CoT-Hybrid	4/10	0.791	0.78	207	154
97	setup_a_bar_for_a_cocktail_party	CoT-FM	0/10	0.805	0.67	671	447
98	laying_tile_floors	SkLoRA-a	6/10	0.863	0.83	596	478
99	sorting_books_on_shelf	Base-700k	0/10	0.064	0.67	387	258
	mean over 50 tasks		139/500	0.495	0.74		

Table 11. **Per-task results, tasks 50–99** (new 2026 tasks).